7 retrieval methods — clinical IR project

Sparse (lexical)

TF-IDF
Term frequency

BM25
Okapi scoring

Vocab overlap required
No GPU needed

Dense (semantic)

SBERT
Dense cosine search

384-dim vectors
Semantic match
CPU ~3-15ms/query

Approximate nearest neighbour

FAISS IVF-Flat
Clustered exact search

FAISS HNSW
Graph-based ANN

FAISS IVF-PQ
Compressed vectors

ANN index types
Scalable to 10M+ docs

Multi-stage pipeline

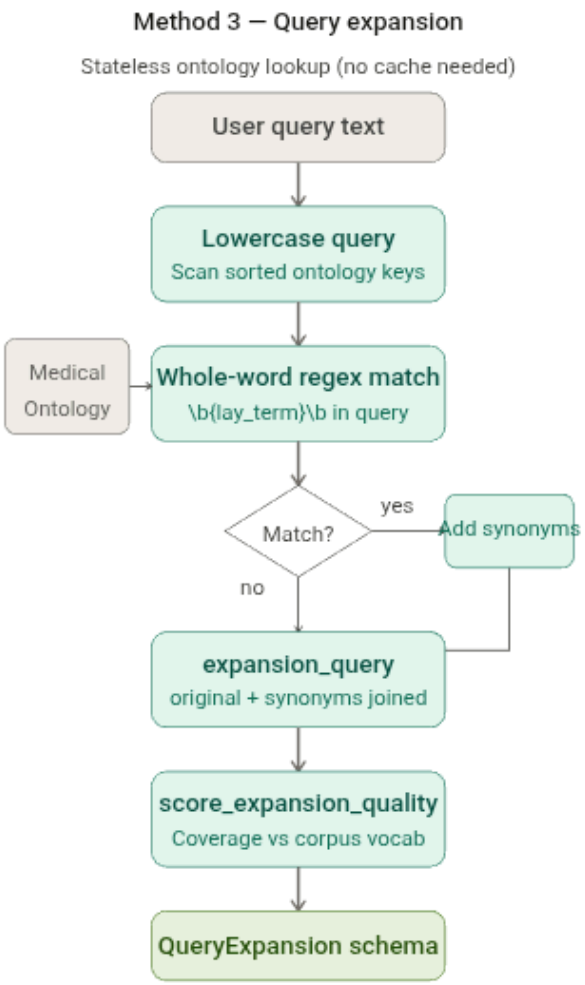
Multi-stage pipeline
Query expansion → BM25 candidates → SBERT re-rank

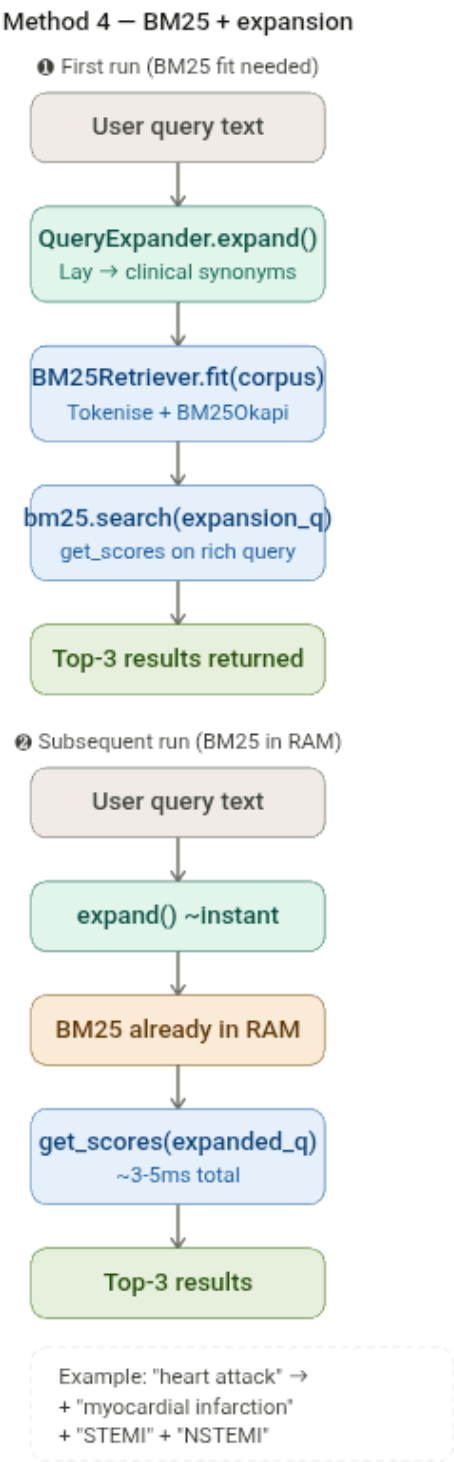
Best accuracy
MRR ~0.65+
15-50ms/query

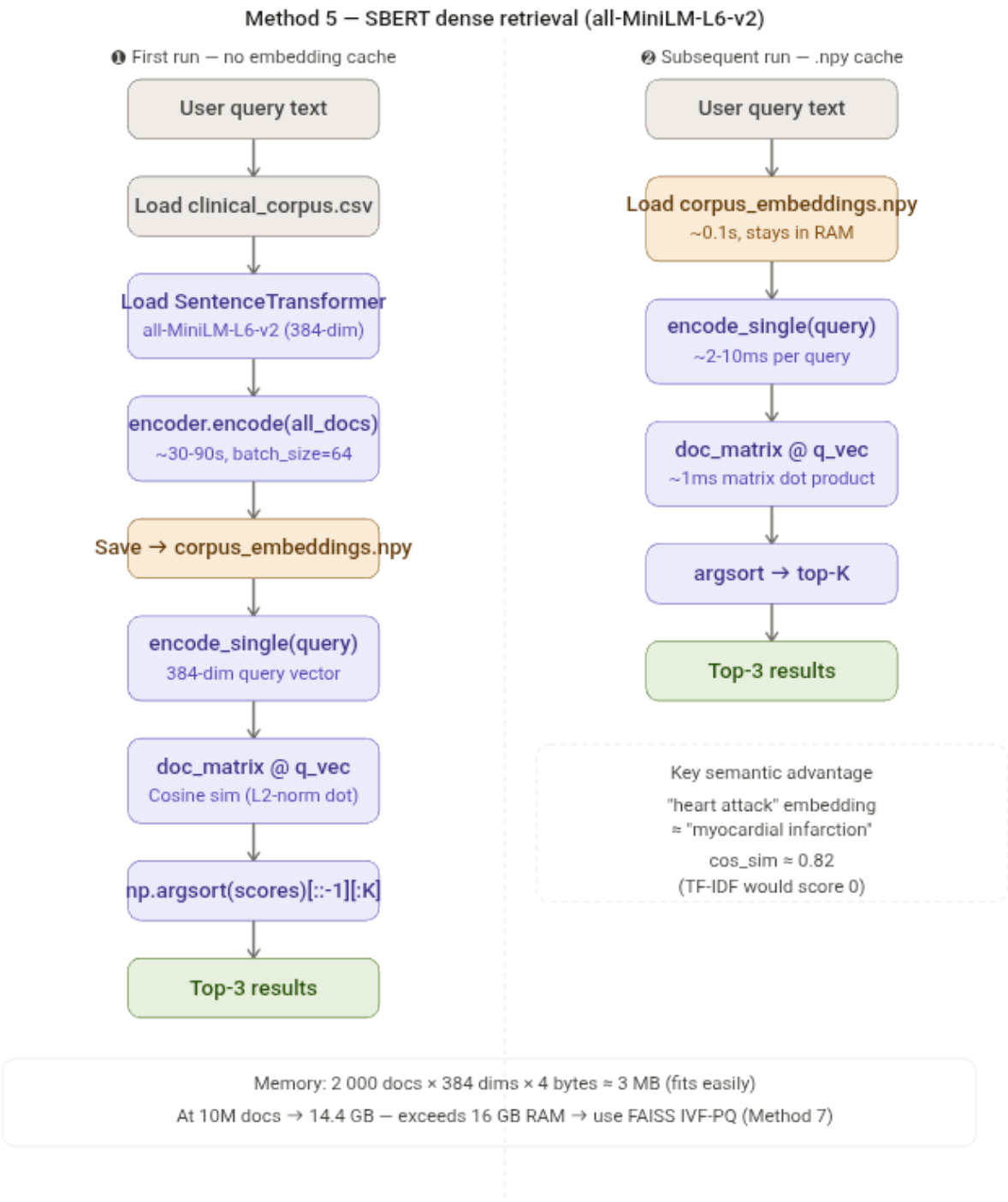
Performance summary (MRR@10)

- TF-IDF / BM25 — MRR ~0.15 (vocabulary mismatch hurts)
- FAISS ANN indexes — same accuracy as SBERT, faster at scale
- SBERT dense — MRR ~0.55-0.65 (semantic understanding)
- Multi-stage pipeline — MRR ~0.65+ (best overall)

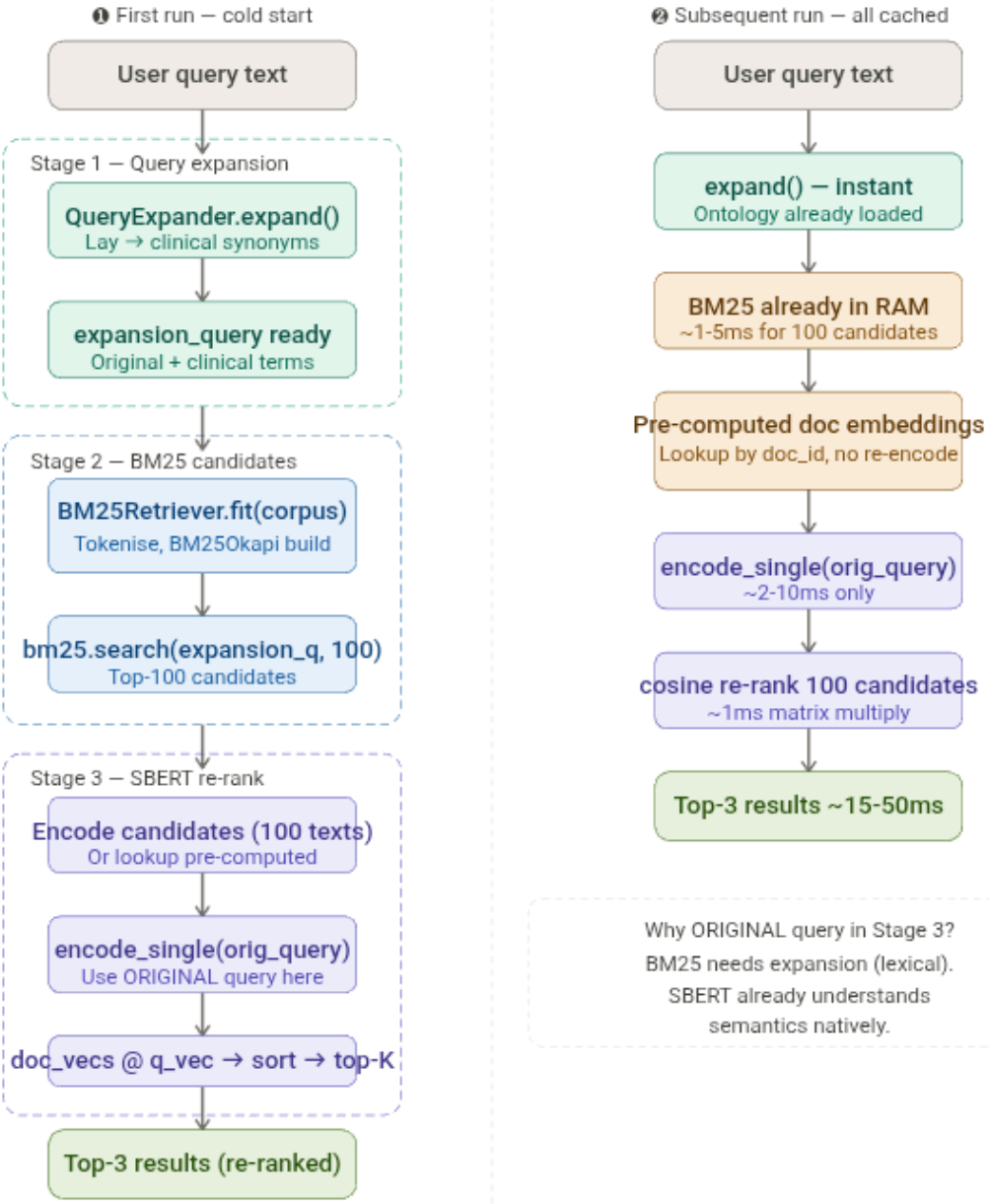
Method 1 — TF-IDF
❶ First run (no cache)
User query text
Load clinical_corpus.csv
TfidfVectorizer.fit_transform
30K features, 1-2 ngrams
Save → tfidf.pkl
vectorizer.transform(query)
Sparse query vector
cosine_similarity
q_vec × doc_matrix
np.argsort(scores)[::-1][:K]
Top-3 results returned
❷ Subsequent run (cache hit)
User query text
Load tfidf.pkl (RAM)
transform(query) ~0.5ms
cosine sim + argsort
Top-3 results
Method 2 — BM25
❶ First run (no cache)
User query text
Load clinical_corpus.csv
Tokenise + BM25Okapi()
k1=1.5, b=0.75, IDF weights
Save → bm25.pkl
query.lower().split()
Tokenise query
bm25.get_scores(tokens)
BM25 score per doc
np.argsort(scores)[::-1][:K]
Top-3 results returned
❷ Subsequent run (cache hit)
User query text
Load bm25.pkl (RAM)
get_scores(tokens) ~2ms
argsort → top-K
Top-3 results


Method 3 — Query expansion
Stateless ontology lookup (no cache needed)
User query text
Lowercase query
Scan sorted ontology keys
Medical Ontology
Whole-word regex match
\b{lay_term}\b in query
Match?
yes
Add synonyms
no
expansion_query
original + synonyms joined
score_expansion_quality
Coverage vs corpus vocab
QueryExpansion schema


Method 4 — BM25 + expansion
❶ First run (BM25 fit needed)
User query text
QueryExpander.expand()
Lay → clinical synonyms
BM25Retriever.fit(corpus)
Tokenise + BM25Okapi
bm25.search(expansion_q)
get_scores on rich query
Top-3 results returned
❷ Subsequent run (BM25 in RAM)
User query text
expand() ~instant
BM25 already in RAM
get_scores(expanded_q)
~3-5ms total
Top-3 results
Example: "heart attack" →
+ "myocardial infarction"
+ "STEMI" + "NSTEMI"


Method 5 — SBERT dense retrieval (all-MiniLM-L6-v2)
❶ First run — no embedding cache
User query text
Load clinical_corpus.csv
Load SentenceTransformer
all-MiniLM-L6-v2 (384-dim)
encoder.encode(all_docs)
~30-90s, batch_size=64
Save → corpus_embeddings.npy
encode_single(query)
384-dim query vector
doc_matrix @ q_vec
Cosine sim (L2-norm dot)
np.argsort(scores)[::-1][:K]
Top-3 results
❷ Subsequent run — .npy cache
User query text
Load corpus_embeddings.npy
~0.1s, stays in RAM
encode_single(query)
~2-10ms per query
doc_matrix @ q_vec
~1ms matrix dot product
argsort → top-K
Top-3 results
Key semantic advantage
"heart attack" embedding
≈ "myocardial infarction"
cos_sim ≈ 0.82
(TF-IDF would score 0)
Memory: 2 000 docs × 384 dims × 4 bytes ≈ 3 MB (fits easily)
At 10M docs → 14.4 GB — exceeds 16 GB RAM → use FAISS IVF-PQ (Method 7)

Method 6 — Multi-stage pipeline

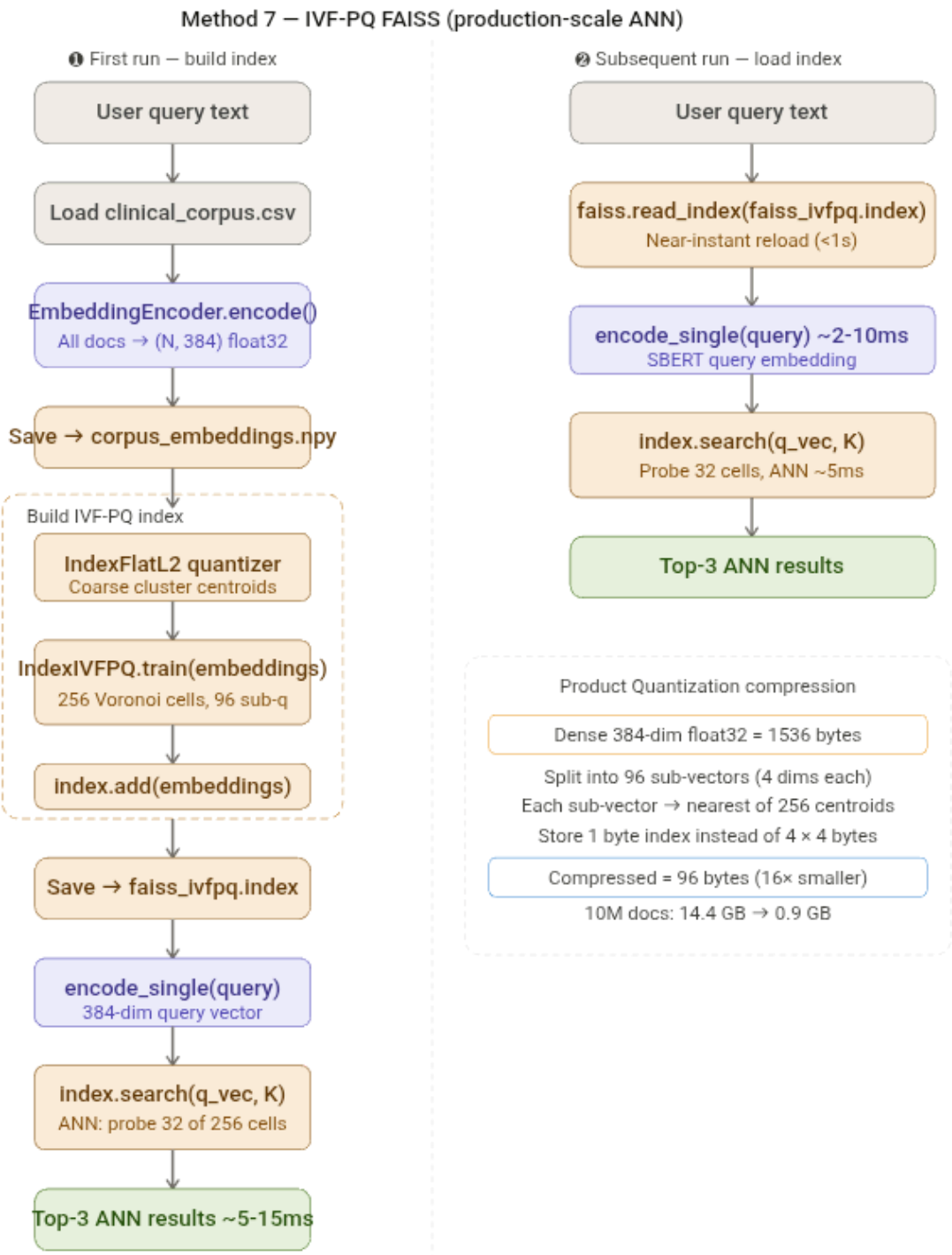


Ablation study — expected ranking: D > C ≥ B > A

A. BM25 only B. BM25+Expansion C. SBERT dense D. MultiStage (full pipeline)

MRR improvement from A→D: ~0.15 → 0.65+

MultiStage = best accuracy + acceptable latency (~15-50ms)


Method 7 — IVF-PQ FAISS (production-scale ANN)
❶ First run — build index
User query text
Load clinical_corpus.csv
EmbeddingEncoder.encode()
All docs → (N, 384) float32
Save → corpus_embeddings.npy
Build IVF-PQ index
IndexFlatL2 quantizer
Coarse cluster centroids
IndexIVFPQ.train(embeddings)
256 Voronoi cells, 96 sub-q
index.add(embeddings)
Save → faiss_ivfpq.index
encode_single(query)
384-dim query vector
index.search(q_vec, K)
ANN: probe 32 of 256 cells
Top-3 ANN results ~5-15ms
❷ Subsequent run — load index
User query text
faiss.read_index(faiss_ivfpq.index)
Near-instant reload (<1s)
encode_single(query) ~2-10ms
SBERT query embedding
index.search(q_vec, K)
Probe 32 cells, ANN ~5ms
Top-3 ANN results
Product Quantization compression
Dense 384-dim float32 = 1536 bytes
Split into 96 sub-vectors (4 dims each)
Each sub-vector → nearest of 256 centroids
Store 1 byte index instead of 4 × 4 bytes
Compressed = 96 bytes (16× smaller)
10M docs: 14.4 GB → 0.9 GB